

Logos Experiment 4: SwiGLU Feed-Forward Network

Controlled alternate-method report for the Logos language-model efficiency project

1. Experiment Aim

The aim of this experiment was to test whether replacing the standard Transformer feed-forward network with a SwiGLU feed-forward network improves the capability-resource tradeoff of the Logos decoder-only language model.

The original baseline used a standard feed-forward network: Linear -> ReLU -> Linear. This experiment replaced that block with a SwiGLU-style gated feed-forward module while keeping the rest of the setup unchanged.

2. Controlled Change

Aspect	Baseline	SwiGLU Experiment
Model family	Decoder-only causal LM	Decoder-only causal LM
Tokenizer	GPT-2 tokenizer	GPT-2 tokenizer
Dataset	OpenWebText subset - 10,000 samples	OpenWebText subset - 10,000 samples
Context length	256 tokens	256 tokens
Layers / heads / embedding	8 layers / 8 heads / 256 dim	8 layers / 8 heads / 256 dim
Normalization	LayerNorm	LayerNorm
Weight tying	No	No
Feed-forward network	Linear -> ReLU -> Linear	SwiGLU gated FFN
Training budget	1500 iterations	1500 iterations

3. What SwiGLU Changes

SwiGLU introduces a gating mechanism inside the feed-forward network. Instead of applying a single activation to one hidden projection, it creates two projections: one gate branch and one value branch. The gate branch uses the SiLU activation and then multiplies the value branch. This gives the model a more flexible way to control which features pass through the FFN.

Baseline FFN:

```
Linear -> ReLU -> Linear
```

SwiGLU FFN:

```
SiLU(gate projection) x value projection -> output projection
```

4. Hypothesis

The hypothesis was that SwiGLU would reduce validation loss and perplexity at roughly the same parameter count, because gated feed-forward layers are commonly stronger than plain ReLU FFNs in modern language-model designs. The expected tradeoff was slightly higher memory or compute cost.

5. Experiment Configuration

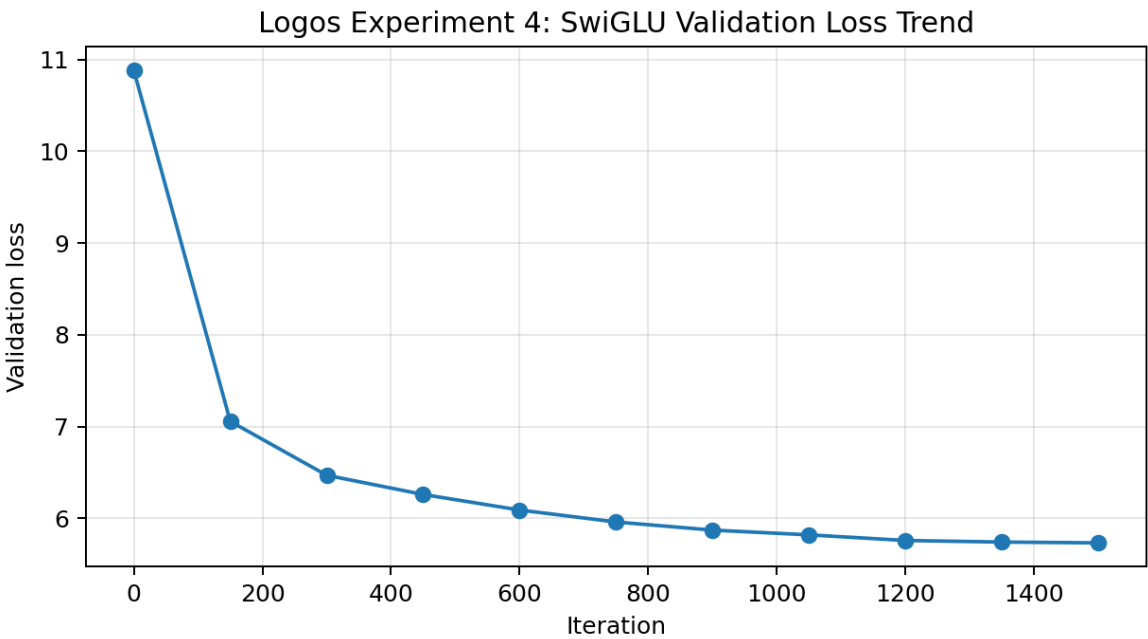
Setting	Value
Run name	logos_exp04_swiglu_ffn
Architecture	Decoder-only GPT-style Transformer
FFN type	SwiGLU
SwiGLU hidden size	682
Parameters	32,158,449
Batch size	64
Block size	256
Layers	8
Attention heads	8
Embedding dimension	256
Dropout	0.2
Optimizer	AdamW
Learning rate	3e-4
Weight decay	0.1
Gradient clipping	1.0
Training iterations	1500

6. Final Results

Metric	SwiGLU Result
Best iteration	1500
Best validation loss	5.7328
Final validation loss	5.7555
Final validation perplexity	315.93
Checkpoint size	122.71 MB
Peak VRAM	11.33 GB
Training time	164.86 sec, about 2.75 min
Training speed	~150,322 tokens/sec
Generation speed	128.61 tokens/sec
Generation latency	7.78 ms/token
Average GPU power	250.28 W
Estimated GPU energy	11.37 Wh

7. Training Behavior

The validation loss steadily decreased from 10.8810 at iteration 0 to 5.7328 at iteration 1500. This indicates stable learning throughout the fixed training budget. The best checkpoint was saved at the final iteration, meaning the model was still improving at the end of the run.



Iteration	Validation loss	Perplexity
0	10.8810	53156.73
150	7.0563	1160.14
300	6.4694	645.10
450	6.2601	523.27

Iteration	Validation loss	Perplexity
600	6.0908	441.77
750	5.9601	387.65
900	5.8718	354.89
1050	5.8201	337.01
1200	5.7588	316.97
1350	5.7416	311.56
1500	5.7328	308.83

8. Comparison with Original Baseline

Metric	Original Baseline	SwiGLU FFN	Result
Parameters	32,159,825	32,158,449	Almost same
Best val loss	5.7716	5.7328	Better with SwiGLU
Final val loss	5.7634	5.7555	Better with SwiGLU
Final perplexity	318.42	315.93	Better with SwiGLU
Checkpoint size	122.71 MB	122.71 MB	Same
Peak VRAM	10.84 GB	11.33 GB	Higher with SwiGLU
Training time	156.42 sec	164.86 sec	Slower with SwiGLU
Training speed	~158k tok/s	~150k tok/s	Slower with SwiGLU
Generation latency	10.20 ms/token	7.78 ms/token	Better with SwiGLU
Energy	10.78 Wh	11.37 Wh	Higher with SwiGLU

9. Interpretation

SwiGLU is the strongest quality-improving alternate method tested so far in the Logos experiment sequence. Unlike GELU and RMSNorm, it improved both best validation loss and final validation loss compared to the original baseline while keeping the parameter count nearly unchanged.

The final perplexity improved from 318.42 to 315.93, and the best validation loss improved from 5.7716 to 5.7328. The generation latency also improved from 10.20 ms/token to 7.78 ms/token in this run.

The tradeoff is that SwiGLU used more VRAM, took slightly longer to train, and consumed slightly more estimated GPU energy. Therefore, it is not a pure efficiency win in every metric, but it is a meaningful capability improvement at nearly identical parameter count.

10. Scientific Conclusion

For the fixed 1500-iteration Logos experiment, the SwiGLU feed-forward network improved language modeling quality compared to the original ReLU FFN baseline. It achieved lower validation loss and lower perplexity while maintaining almost the same parameter count and checkpoint size. The cost was slightly higher VRAM, slower training throughput, and higher energy use.

Conclusion: SwiGLU is a successful quality-focused architectural improvement for Logos, but it should be combined with a size-saving method such as weight tying to test whether quality and parameter efficiency can be improved together.

11. Next Recommended Experiment

The next scientific experiment should combine the two successful directions so far: weight tying and SwiGLU.

Next experiment	Purpose
Weight Tying + SwiGLU	Test whether strong parameter/storage efficiency from weight tying can be combined w

Expected question: Can Logos keep the smaller 19M-parameter structure from weight tying while gaining the lower validation loss of SwiGLU?